

Anomaly Detection for CNP Transactions: A Budget-Aware, Chronologically Validated Approach

Anna Kravchenko

Supervised Research (Law, Economics, and Data Science), 851-0763-00L HS2025

akravchenko@student.ethz.ch, ETH Zurich

Instructors: Prof. Elliott Ash, Prof. Sergio Galletta; Teaching Assistant: Benjamin Kohler

Abstract

In practice, card-not-present (CNP) fraud detection is constrained by limited reviewer capacity and delayed labels, yet most academic work still optimizes ROC-AUC on randomly shuffled data, a weak proxy for what a review team actually sees. This project reframes fraud detection as anomaly ranking under a fixed daily alert budget, using the IEEE-CIS/Vesta dataset (590,540 labeled card-not-present transactions) with strictly chronological train, validation, and holdout windows. Two unsupervised detectors, an Isolation Forest and a small tabular autoencoder, are trained without ever using the fraud label, and a single calibrated LightGBM model is included only as a labeled reference point rather than the object of optimization. Evaluation centers on Precision@K and Recall@K at realistic daily review budgets (K = 50, 100, 200, and fixed alert-rate variants), alongside PR-AUC, weekly temporal-stability checks, and SHAP-based explanations for individual alerts. The calibrated reference reaches a holdout PR-AUC of 0.485, against 0.062 and 0.073 for the Isolation Forest and autoencoder respectively, but the autoencoder's top-K alerts overlap with the GBM's by only 11-22% and independently catch 76-96 additional true frauds, motivating a blended alert-budget policy rather than a single best model.

1. Introduction

Card-not-present (CNP) transactions continue to grow as commerce moves increasingly online, and though still a small share of overall transaction volume, they generate material losses and customer friction when misclassified. Fraud monitoring operates under two binding constraints most academic benchmarks ignore: reviewer capacity is limited, forcing any model into a small number of high-value alerts per day, and labels arrive late or are sparse, since confirmed fraud depends on chargebacks and manual investigation. Classic objectives such as ROC-AUC are a weak proxy for this reality: they ignore alert budgets, are insensitive to the severe class imbalance in fraud data, and can look strong even when the top of the ranked list, the part a review team actually sees, underperforms.

This project treats fraud detection as anomaly ranking with top-K alerting rather than as a standard classification task. The central research question is: under a fixed daily review budget and strictly time-respecting validation, how well can unsupervised anomaly detectors identify fraudulent card-not-present transactions, and how much does a labeled supervised model add on top of that? The focus throughout is on (a) alert quality at realistic review budgets, (b) the resulting workload in alerts per 1,000 transactions and estimated analyst-hours, (c) robustness to the drift documented across the dataset's roughly six-month span, and (d) reviewer-facing explanations that make individual alerts defensible rather than opaque scores.

The pipeline spans the IEEE-CIS/Vesta CNP fraud dataset: exploratory analysis and a feature audit, a strictly chronological 60/20/20 train/validation/holdout split, and two parallel preprocessing branches feeding an Isolation Forest, a small tabular autoencoder, and a single isotonic-calibrated LightGBM model included only as a labeled reference. On the holdout window, the calibrated reference reaches PR-AUC 0.485 against 0.062 (Isolation Forest) and 0.073 (autoencoder), a large gap, as expected given that the reference model is the only one using the fraud label. Precision@K/Recall@K tables and curves translate these scores into daily alert-budget terms, and a GBM-versus-autoencoder overlap analysis shows the two models flag largely different transactions, with the autoencoder-only slice independently containing dozens of frauds the GBM's own alert list misses entirely.

These findings echo a recurring theme in the credit-card and CNP fraud literature: concept drift, class imbalance, and delayed verification make randomly split, AUC-optimized evaluation an unreliable guide to production value [1, 2]. This project differs from most of the reviewed literature by reporting results in operational terms, alerts per day, analyst-hours, and precision at specific budgets, rather than a single discrimination score, and by treating the supervised model as a reference rather than an optimization target, keeping results relevant even when labels are scarce.

2. Literature Review

Concept drift and verification latency. This project's design was directly shaped by three recurring themes emphasized in the reviewed credit-card and CNP fraud detection literature. Dal Pozzolo et al. [1] show that realistic evaluation must account for concept drift, extreme class imbalance, and verification latency, meaning that only a small, delayed subset of transactions ever receives a confirmed label; ignoring these leads to inflated, non-reproducible performance estimates. Carcillo et al. [2] build on this with a streaming active-learning framework for real-life fraud detection, explicitly designed around the reality that analyst review capacity, not model capacity, is often the binding constraint, the same premise behind this project's top-K, budget-first evaluation.

Supervised and hybrid approaches. A second strand emphasizes strong supervised or hybrid models evaluated on largely static, randomly split benchmarks: Nguyen et al. [3] combine CatBoost with a deep neural network for card fraud detection, Tian et al. [4] propose an adaptive graph neural network that models transactions relationally, and Manoharan et al. [5] survey machine-learning approaches to e-commerce fraud more broadly. These papers report strong global discrimination metrics but generally do not report alert-budget-specific precision/recall or analyst workload, and most do not enforce chronological validation as strictly as this project does.

Unsupervised and label-light methods. A third, more recent strand moves toward unsupervised and label-light methods: Li et al. [6] use contrastive learning for unsupervised fraud detection in e-commerce, directly motivating this project's choice to make unsupervised anomaly detectors, rather than a supervised classifier, the primary object of study. The specific unsupervised methods used here, Isolation Forest [7] and autoencoder reconstruction error [8], are established baselines within that literature rather than novel architectures; the contribution here is less a new detector and more a budget-aware, chronologically valid evaluation protocol applied to them, plus a reviewer-facing explanation layer built on Molnar's [9] framework for interpretable machine learning via SHAP.

Relative to this literature, the project's departure is evaluative rather than architectural: instead of reporting a single ROC-AUC or F1 score on a randomly shuffled test set, every result here is reported per day, on a strictly future holdout window, at a small number of realistic alert budgets, with an explicit translation into analyst-hours. This responds to the drift and verification-latency concerns raised by Dal Pozzolo et al. [1]: drift, through strictly time-

respecting evaluation and the weekly stability checks in Section 5; verification latency, through the choice to make label-free unsupervised detectors the primary object of study rather than a label-dependent classifier.

3. Data and Summary Statistics

The project uses the IEEE-CIS/Vesta Fraud Detection dataset [10], a real (non-synthetic) collection of card-not-present transactions released by Vesta Corporation for an official IEEE Computational Intelligence Society challenge hosted on Kaggle. Only the labeled training tables are used: `train_transaction.csv` (590,540 rows, with the binary target `isFraud`, transaction amount, product/card/address fields, match flags, and 339 anonymized V-columns) and `train_identity.csv` (144,233 rows of device, browser, and network features), joined on `TransactionID`. Identity information is available for only 24.4% of transactions. The unlabeled test files are excluded from every analysis, since they carry no ground-truth label and cannot support evaluation or chronological validation.

`TransactionDT` records elapsed time from a hidden reference point rather than a calendar date; it is converted into an `event_time` column used to reconstruct chronological order. A strictly time-based split at the 60th and 80th percentile of `event_time` yields a train window of 354,324 transactions, a validation window of 118,108, and a holdout window of 118,108, spanning a little under six months. Fraud rates are 3.38% (train), 3.90% (validation), and 3.44% (holdout), close enough across windows that the split is usable for model development without one window being systematically easier or harder than the others.

The fraud rate itself is not stationary. Figure 1 plots the weekly fraud rate across the full observed window: it ranges from roughly 2.1% to 5.1%, with a pronounced one-week spike in late March 2018 and no single stable baseline. This single fact motivates most of the project's methodological choices, chronological rather than random splitting, explicit drift diagnostics via the population stability index (PSI), and per-day rather than pooled evaluation.



Figure 1. Weekly fraud rate, full observed window.

Missingness is structural rather than random: several `id_*` and `D*/dist*` fields are missing in 85-99% of rows, and fields present tend to show materially different fraud rates than the same fields when missing, so missingness is treated as signal via explicit indicator flags rather than silently imputed away. Figure 2 visualizes this by tracking the weekly missing rate for the 60 columns with the highest overall missing share, clustered by how similarly their missing rate moves over time: most of these columns stay consistently missing (85-99%+) for almost the entire window, but several `id_*/V-block` columns show a markedly lower missing rate during the first few weeks of observation before settling into the same high, stable rate from January onward, indicating a shift in data collection or schema early in the window rather than a fixed, unchanging pattern.

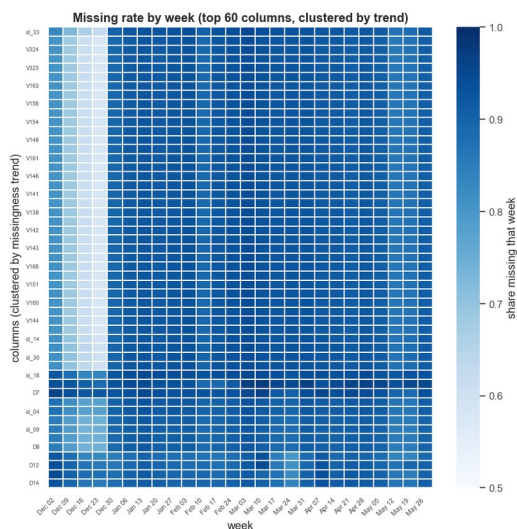


Figure 2. Weekly missing rate for the 60 columns with the highest overall missing share, clustered by missingness trend (darker = higher share missing that week).

TransactionAmt is heavily right-skewed (mean $\approx$ \$135, median $\approx$ \$69, 99th percentile $\approx$ \$1,104, maximum $\approx$ \$31,937). Table 1 bins TransactionAmt into deciles and reports each bin’s fraud rate and share of total fraud-dollar losses: the highest-amount decile alone accounts for 54.4% of total fraud losses even though its fraud rate (5.1%) is not the highest of the ten bins; that distinction belongs to the lowest-amount decile (5.6%). Fraud is therefore both relatively common among small transactions and concentrated in dollar terms among large ones, two different senses of “risky” that a single fraud-rate number would blur together.

The middle deciles are comparatively quiet: fraud rate dips to around 1.9-3.2% for transactions between roughly \$35 and \$58, and their combined share of total fraud losses stays under 5%, so most of the operational risk sits at the two tails of the amount distribution

rather than in the bulk of everyday transaction sizes. This U-shaped pattern echoes the day-of-week and hourly fraud-rate patterns documented in the EDA notebook: risk concentrates in specific, identifiable slices of the data rather than spreading uniformly, reinforcing the case for score-based ranking over a single fixed threshold.

Table 1. Fraud rate and share of total fraud losses by TransactionAmt decile.

Amount range (\$)	Count	Fraud rate	Loss share
0.25 – 25.95	59,511	5.6%	1.8%
25.95 – 35.95	61,650	3.2%	2.0%
35.95 – 49.00	65,116	3.2%	3.0%
49.00 – 57.95	59,647	1.9%	2.0%
57.95 – 68.77	49,346	2.9%	2.8%
68.77 – 100.00	73,349	3.6%	7.4%
100.00 – 117.00	72,079	2.0%	5.2%
117.00 – 159.95	32,399	4.3%	6.5%
159.95 – 275.29	58,390	3.8%	15.0%
275.29 – 31,937.39	59,053	5.1%	54.4%

High-cardinality categorical fields, DeviceInfo alone has about 1,786 unique values, make raw one-hot encoding impractical and motivate the frequency-encoding scheme used in preprocessing.

4. Methods

Figure 3 summarizes the end-to-end pipeline. Raw labeled data is profiled and audited, split chronologically before any statistic is computed, passed through a leakage-safe preprocessing stage that produces two parallel feature representations, and finally scored by three models whose outputs feed a budget-aware evaluation and a final staffing recommendation. Every fitted statistic, rare-category vocabularies, frequency counts, UID aggregates, PCA components, imputation medians, scaler mean/std, is fit on the training window only and applied transform-only to validation and holdout rows.

This train-only fitting rule is what keeps the holdout evaluation honest: every encoding, imputation value, and scaling parameter is frozen before the validation or holdout windows are ever touched, so no information from the future leaks backward into how a feature is represented. The two branches exist because the three models need different feature geometry: LightGBM splits on raw feature values and is largely indifferent to categorical encoding scheme or scale, while the Isolation Forest’s random splits and the autoencoder’s distance-based reconstruction loss are both sensitive to feature scale and to how sparse one-hot categories inflate input dimensionality, motivating separate branches rather than forcing every model through one shared representation.

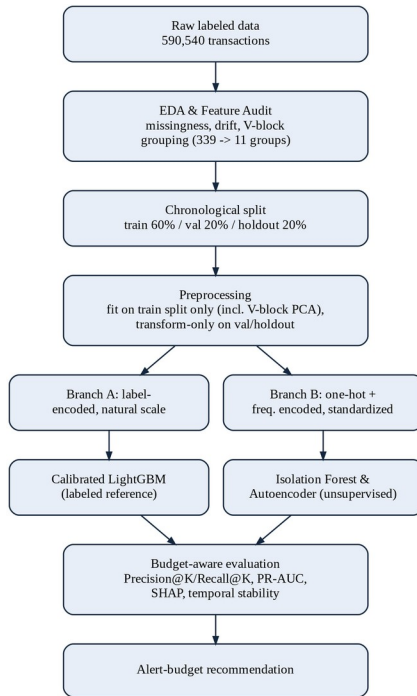


Figure 3. Pipeline overview.

Feature engineering follows directly from the data audit. The 339 raw V-columns, found to cluster into 11 groups by missingness pattern, are each reduced to their first principal component fit on the training split. High-cardinality identifiers (DeviceInfo, email domains, several `id_*` fields) are frequency-encoded rather than one-hot encoded, using counts computed on the training window only; categories unseen in training map to a count of zero. Categorical fields with adequate frequency ($\geq 0.1\%$ of training rows) keep their own level; rarer values collapse into an explicit `__OTHER__` bucket, and missing values get their own `__MISS__` label. Composite UID features (card/address/email combinations approximating a persistent customer or device entity) receive aggregate statistics, the mean and standard deviation of that entity’s TransactionAmt in the training window. Explicit missingness indicators are added only for columns whose training-window missing share falls in an informative 5-95% band.

Two robustness diagnostics from the feature audit motivate and validate this design. A leakage smoke test scores each candidate feature alone on the validation split; the highest single-feature AUCs (dist2: 0.553, TransactionAmt: 0.506, card2: 0.496) sit well below the level that would flag a hidden join or label leak. Separately, a population stability index (PSI) audit compares each feature’s monthly distribution to its first observed month, and train-vs-validation distributions directly ($\text{PSI} < 0.10$ = negligible shift, $0.10\text{-}0.25$ = moderate, > 0.25 = major). Most features are stable: train-vs-validation

PSI is 0.000-0.014 for TransactionAmt, card1, card2, and addr1, but a few D-block time-delta features drift more: D4 reaches a maximum monthly PSI of 0.209 and D1 reaches 0.155, both moderate shifts. These features are kept, since dropping them would discard real signal, but their drift is exactly why every split in this project is chronological rather than random, and why the weekly stability check described below re-examines the trained models rather than assuming the audit-stage findings still hold once feature engineering and modeling are complete.

Because label-encoding assigns arbitrary integer codes to categories — an encoding that tree-based models like LightGBM [11] ignore but that distance- or reconstruction-based models would misread as a real ordinal scale, two parallel feature branches are built from the same engineered columns. Branch A (192 model features, after excluding the identifier, label, and split-bookkeeping columns) label-encodes categoricals and leaves numeric features on their natural scale, feeding the calibrated LightGBM reference. Branch B (373 model features after one-hot expansion) instead one-hot encodes low-cardinality categoricals, keeps high-cardinality fields as their already leakage-safe frequency encoding, and median-imputes plus standardizes every numeric feature, feeding the two distance/reconstruction-based unsupervised models. Both branches fit every encoder and scaler on training-split rows only.

Three models are trained. The Isolation Forest (scikit-learn [12], `random_state=42`) is fit on Branch B training rows only, with `isFraud` never used at any point; a small grid of 5 configurations varying `n_estimators` (100/150/200), `max_samples` (32,768/65,536 rows per tree), and contamination is evaluated by validation PR-AUC, and the best configuration (`n_estimators=150`, `max_samples=32,768`, `contamination='auto'`) is kept. Its anomaly score is the negative of scikit-learn’s `score_samples` output, so higher values consistently mean “more anomalous” across all three models. The autoencoder is a small bottleneck multilayer perceptron ($373 \rightarrow 32 \rightarrow 8 \rightarrow 32 \rightarrow 373$), implemented with scikit-learn’s `MLPRegressor` (ReLU activation, Adam solver, batch size 4,096, `random_state=42`) trained via `fit(X, X)` rather than a dedicated deep-learning framework; a small grid over hidden-layer width, training epochs, and learning rate is likewise evaluated by validation PR-AUC, and the best configuration ($32 \rightarrow 8 \rightarrow 32$, 15 epochs, learning rate $1e-3$) is kept. Its anomaly score is the per-row mean squared reconstruction error, again computed without ever touching the label. Both unsupervised models are selected on the validation split only; the holdout window is never used for model selection. The

calibrated LightGBM reference is the only model that uses isFraud: it is trained on Branch A with early stopping (100-round patience against the validation AUC, within a 1,500-round budget; the model stopped at 344 rounds), then isotonicly calibrated by fitting a monotonic mapping from raw score to probability on validation predictions against validation labels. This model is deliberately kept singular and clearly labeled as a reference comparator rather than blended with the unsupervised scores.

Evaluation is built around the holdout window (42 days, 118,108 transactions, base fraud rate 3.44%), the only split whose labels were never used for any fitting or selection decision. For each day, transactions are ranked by score and the top K are treated as that day’s alerts; Precision@K is the share of those alerts that are true fraud, Recall@K is the share of that day’s total fraud they capture. Results are reported both for fixed K (50, 100, 200) and for a fixed alert-rate variant (0.5%, 1%, 2% of that day’s volume). Two operational translations accompany every precision/recall number: alerts per 1,000 transactions, and estimated analyst-hours per day assuming 8 minutes of review time per alert. PR-AUC is used as the primary ranking-quality metric, since it is far more informative than ROC-AUC at a 3.4% base rate; ROC-AUC is retained only as a secondary, descriptive number. Model behavior is checked for temporal robustness by splitting the holdout window into its six constituent weeks and tracking Precision@100 and the gap between mean calibrated score and observed fraud rate week by week. Explanations for the calibrated reference are computed with SHAP’s [13] TreeExplainer on a 4,000-row holdout sample, producing both global feature-importance rankings and short case notes for individual high-scored alerts, following Molnar’s [9] framework for making model output locally defensible to a human reviewer. Finally, a GBM-versus-autoencoder overlap analysis at K = 50/100/200 checks how much of each model’s alert set is unique, and whether the autoencoder-only alerts contain real fraud the GBM’s list misses.

5. Results

Table 2 reports holdout discrimination for all three models. The calibrated LightGBM reference reaches holdout AUC 0.905 and PR-AUC 0.485, far ahead of the Isolation Forest (AUC 0.689, PR-AUC 0.062) and the autoencoder (AUC 0.701, PR-AUC 0.073), expected, since it is the only model that ever sees the fraud label. Both unsupervised detectors still clear the 3.44% base rate by roughly 1.8-2.1x in PR-AUC terms using no label information whatsoever, and the autoencoder outperforms the Isolation Forest throughout, plausibly because its nonlinear

reconstruction can capture feature interactions the axis-aligned Isolation Forest partly misses.

Table 2. Holdout discrimination by model.

Model	Val AUC	Hold. AUC	Hold. PR-AUC
Isolation Forest	0.708	0.689	0.062
Autoencoder	0.724	0.701	0.073
GBM (reference)	0.933	0.905	0.485

Figure 4 checks whether isotonic calibration actually improved probability accuracy: within each predicted-probability decile on the holdout set, does the mean predicted probability match the observed fraud rate? Raw LightGBM probabilities are already close to calibrated in most bins, and isotonic calibration does not help uniformly: in the top decile it is mildly overconfident (predicted 0.311 vs. observed 0.267), while the raw score was actually closer there (0.244 vs. 0.238). Across the lower and middle deciles, though, the calibrated curve tracks the diagonal more tightly than the raw score.

The decile-level values also show where isotonic calibration actually does its work: raw and calibrated predicted probabilities differ by at most 0.01 across deciles 1-8, and only diverge meaningfully in deciles 9-10, where the calibrated model pushes its mean prediction up from 0.028 to 0.054 (decile 9) and from 0.244 to 0.311 (decile 10). This concentration makes sense operationally, alert prioritization only depends on how transactions are ranked near the top of the score distribution, which is exactly the region where the raw score was least trustworthy and where recalibration has the most room to act, while leaving the bulk of the distribution essentially untouched.

Table 3 lists the exact per-decile values underlying Figure 4.

Table 3. Calibration reliability by predicted-probability decile: raw vs. isotonic-calibrated LightGBM (holdout).

Decile	Raw pred.	Raw obs.	Cal. pred.	Cal. obs.
1	0.001	0.002	0.001	0.002
2	0.002	0.002	0.002	0.002
3	0.002	0.002	0.002	0.003
4	0.003	0.004	0.004	0.005
5	0.005	0.005	0.005	0.005
6	0.006	0.007	0.007	0.009
7	0.009	0.015	0.011	0.016
8	0.014	0.025	0.022	0.027
9	0.028	0.045	0.054	0.049
10	0.244	0.238	0.311	0.267

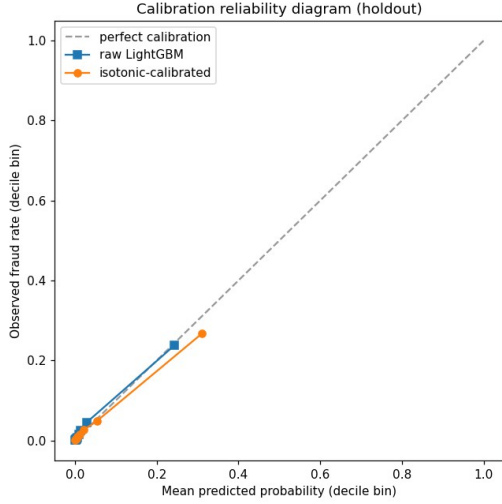


Figure 4. Calibration reliability: mean predicted probability vs. observed fraud rate, by holdout decile.

Table 4 translates these scores into the fixed-K alert-budget terms the project is built around. At a 50-alert daily budget, the calibrated reference reaches 67.3% precision and 35.3% recall, versus 7.7% / 4.1% (Isolation Forest) and 12.1% / 6.5% (autoencoder). Widening the budget to $K = 200$ roughly doubles the reference’s recall (62.6%) while precision falls to 30.3%; the unsupervised detectors stay comparatively flat across the same range (about 8-12% precision regardless of K).

Table 4. Precision@K / Recall@K on the holdout window.

Model	K	P@K	R@K	Alerts/1k	Hrs/day
IF	50	0.077	0.041	17.8	6.7
IF	100	0.076	0.078	35.6	13.3
IF	200	0.080	0.167	71.1	26.7
AE	50	0.121	0.065	17.8	6.7
AE	100	0.110	0.114	35.6	13.3
AE	200	0.102	0.210	71.1	26.7
GBM	50	0.673	0.353	17.8	6.7
GBM	100	0.469	0.486	35.6	13.3
GBM	200	0.303	0.626	71.1	26.7

Figure 5 extends the three point estimates in Table 4 to the full budget range. The calibrated reference’s precision decays smoothly from about 0.88 at $K = 10$ toward the base rate as K grows into the hundreds. Isolation Forest stays roughly flat and low throughout (about 7-8% precision), while the autoencoder starts higher (about 16% at $K = 10$) and declines toward the same low range as K grows into the hundreds. Figure 6 reframes the same result against estimated analyst-hours per day rather than the abstract K : doubling the budget from $K = 10$ to $K = 20$ (1.3 to 2.7 analyst-hours/day) roughly doubles the reference’s recall as well (0.095 to 0.184), while the same budget increase barely moves either unsupervised detector.

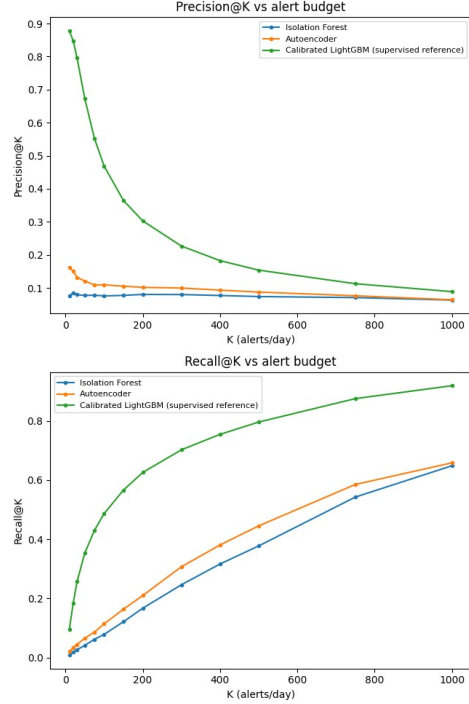


Figure 5. Precision@K (top) and Recall@K (bottom) across the full alert-budget range.

It is worth separating budget size from model choice here: alert volume and analyst-hours in Table 4 depend only on K , not on which model is ranking, so the same 71.1 alerts per 1,000 transactions and 26.7 hours per day at $K = 200$ apply regardless of whether the calibrated reference or either unsupervised detector is doing the ranking. What changes with model choice is only the yield of that fixed workload, how many of those alerts are actually fraud, which is exactly what Precision@K captures.

Taken together, Figures 5 and 6 show that the autoencoder’s early precision advantage over the Isolation Forest at very small budgets (about 16% vs. 8% at $K = 10$) erodes steadily as the budget widens: the gap narrows to about 2 points by $K = 200$ and to under a point by $K = 750$. This means the practical choice between the two unsupervised detectors matters far less than the choice of budget itself: at a lean daily allowance the autoencoder is the better single ranking to lead with, but at a wider budget either detector delivers a similar, modest lift over the base rate, and the more consequential decision is how much of that budget to allocate to the calibrated reference versus either anomaly detector. In practice, this argues for defaulting to the calibrated GBM wherever a labeled reference is available, and sizing the unsupervised layer to whatever reviewer capacity remains, rather than treating the two anomaly detectors as competing primary rankings.

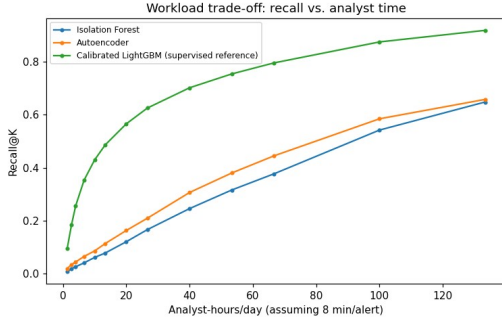


Figure 6. Workload trade-off: Recall@K vs. estimated analyst-hours/day.

Across the six constituent weeks of the holdout window, the calibrated reference’s Precision@100 ranges 0.41-0.53 with no clear downward trend, and its calibration gap stays within about ± 0.7 percentage points every week (Figure 7).

Table 5 lists the exact per-week values underlying Figure 7.

Table 5. Weekly Precision@100 and calibration gap on the holdout window.

Week	N days	Precision@100	Calibration gap
0	7	0.407	-0.0021
1	7	0.489	+0.0043
2	7	0.407	+0.0072
3	7	0.461	+0.0066
4	7	0.529	-0.0001
5	7	0.520	-0.0002

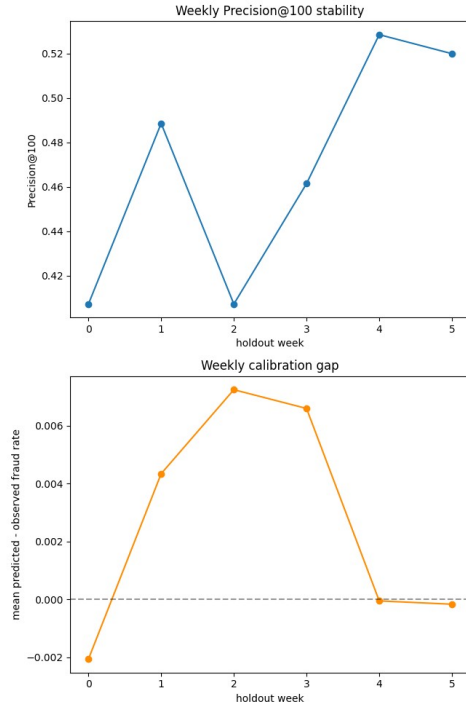


Figure 7. Weekly Precision@100 stability (top) and calibration gap (bottom) across the six holdout weeks.

Figure 8 shows the SHAP beeswarm plot for the calibrated reference: C5, C13, TransactionDT, TransactionAmt, and C1 are the top individual drivers of the model’s scores, consistent with features flagged as informative in the exploratory analysis. C1 shows the clearest directional pattern: high values (red) push the score sharply upward for a subset of transactions, while low values (blue) stay close to baseline; C5 and C13 instead show mixed high/low coloring on both sides, meaning their relationship with the model’s output is not simply higher-value-higher-risk.

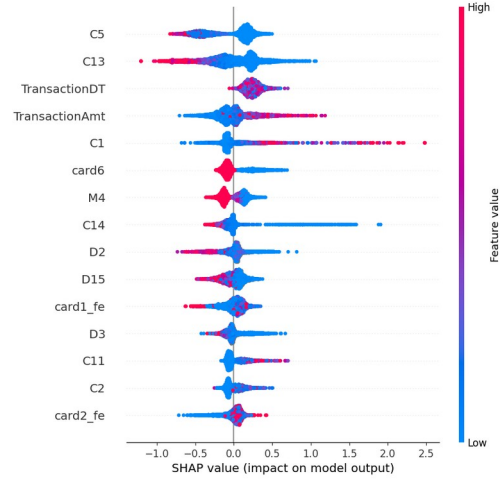


Figure 8. SHAP beeswarm plot for the calibrated LightGBM reference (holdout sample).

Most importantly for the alert-budget design, the calibrated GBM’s and the autoencoder’s top-K alert sets overlap by only 11-22% across $K = 50/100/200$, and the autoencoder-only slice independently contains 76, 96, and 96 additional true-fraud transactions at those three budgets respectively, cases the GBM’s own alert list would have missed entirely (Table 6).

Table 6. GBM vs. Autoencoder alert overlap by budget K.

K	Overlap %	AE-only alerts/day	AE-only frauds found
50	11.3%	44.4	76
100	15.8%	84.2	96
200	22.2%	155.7	96

This motivates the project’s final recommendation: fill the daily alert budget primarily from the calibrated GBM ranking, but reserve roughly 10-20% of it for transactions the autoencoder ranks highly but the GBM does not. Table 7 translates this into two illustrative staffing scenarios: at a lean-team budget of about 15 alerts/day, the calibrated reference reaches 87.5% precision versus 8.7% (Isolation Forest) and 15.7% (autoencoder); widening to an expanded-team budget of about 30 alerts/day lowers the reference’s

precision to 79.6% while roughly doubling recall (14.2% to 25.6%).

Table 7. Alert-budget recommendation for two staffing scenarios.

Scenario	Model	K	P@K	R@K	Hrs/day
Lean (~15/day)	Isolation Forest	15	0.087	0.015	2.0
Lean (~15/day)	Autoencoder	15	0.157	0.028	2.0
Lean (~15/day)	GBM (reference)	15	0.875	0.142	2.0
Expanded (~30/day)	Isolation Forest	30	0.079	0.027	4.0
Expanded (~30/day)	Autoencoder	30	0.132	0.044	4.0
Expanded (~30/day)	GBM (reference)	30	0.796	0.256	4.0

6. Conclusion

Under strictly chronological validation and a realistic daily alert-budget framing, unsupervised anomaly detectors provide a modest but genuine, label-free lift over the base fraud rate, while a calibrated supervised reference is substantially stronger whenever labels are available, reaching 67-88% alert precision at realistic budgets against roughly 8-12% for the unsupervised models. This advantage holds up over time rather than reflecting a single favorable period: across the six holdout weeks, the calibrated reference’s Precision@100 stays within 0.41-0.53 with no clear downward trend. The two models are not simply substitutes, however: the autoencoder’s alerts overlap with the GBM’s by only 11-22% and independently surface dozens of frauds the GBM’s own ranking never would have, supporting a blended alert policy rather than a single best model. SHAP-based explanations further ground individual alerts in specific feature values, notably C5, C13, TransactionDT, TransactionAmt, and C1, rather than leaving them as opaque scores, one practical step toward the interpretability that budget-constrained review teams need.

Several limitations bound these conclusions. All reported metrics are point estimates: no bootstrap confidence intervals or significance tests accompany the AUC, PR-AUC, or Precision@K/Recall@K comparisons, so small gaps between models, particularly between the Isolation Forest and the autoencoder, should be read as suggestive rather than statistically confirmed at this 3.44% base rate. The temporal-stability check covers only six weeks of holdout data, too short to rule out slower seasonal drift; the 8-minutes-per-alert review-time assumption is illustrative rather than measured; and the

unsupervised models’ hyperparameters were tuned only over a small validation-selected grid rather than an exhaustive or Bayesian search. Future work could extend the stability check to a longer rolling-window backtest, add bootstrap confidence intervals around the headline metrics, formally tune the blended-budget policy on a validation window, and explore graph- or contrastive-learning-based unsupervised detectors [4, 6] on top of the UID entity features already engineered here.

Data and Code Availability

A full reproducible pipeline (four Jupyter notebooks covering EDA, feature audit, preprocessing, and modeling; a README with setup instructions; requirements.txt; and all fitted artifacts, processed features, and prediction outputs) accompanies this paper as the replication package, available at <https://github.com/akrav4enk0/cnp-fraud-anomaly-detection>. Raw competition data is not redistributed due to Kaggle’s data-usage terms but is obtainable directly from <https://www.kaggle.com/competitions/ieee-fraud-detection/data>.

References

- [1] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi. Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8):3784–3797, 2018.
- [2] F. Carcillo, Y.-A. Le Borgne, O. Caelen, and G. Bontempi. Streaming active learning strategies for real-life credit card fraud detection: Assessment and visualization. *arXiv:1804.07481*, 2018.
- [3] N. Nguyen, T. Duong, T. Chau, V.-H. Nguyen, T. Trinh, D. Tran, and T. Ho. A proposed model for card fraud detection based on CatBoost and deep neural network. *IEEE Access*, 10:96852–96861, 2022.
- [4] Y. Tian, G. Liu, J. Wang, and M. Zhou. Transaction fraud detection via an adaptive graph neural network. *arXiv:2307.05633*, 2023.
- [5] G. Manoharan, S. D. N. H. Ali, M. Sathe, A. Karthik, A. Nagpal, and A. Sidana. Fraud detection in e-commerce transactions: A machine learning perspective. In *Proc. Int. Conf. on Advances in Computing, Communication and Applied Informatics (ACCAI)*, 2024.
- [6] X. Li, Y. Peng, X. Sun, Y. Duan, Z. Fang, and T. Tang. Unsupervised detection of fraudulent transactions in e-commerce using contrastive learning. *arXiv:2503.18841*, 2025.

- [7] F. T. Liu, K. M. Ting, and Z.-H. Zhou. Isolation forest. In Proc. IEEE Int. Conf. on Data Mining, pages 413–422, 2008.
- [8] R. Chalapathy and S. Chawla. Deep learning for anomaly detection: A survey. arXiv:1901.03407, 2019.
- [9] C. Molnar. Interpretable Machine Learning: A Guide for Making Black Box Models Explainable. 2nd edition, 2022.
- [10] A. Howard, B. Bouchon-Meunier, C. Huang, inversion, J. Lei, Lynn@Vesta, Marcus2010, and H. Abbass. IEEE-CIS Fraud Detection. Kaggle, 2019.
- [11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In Advances in Neural Information Processing Systems 30, pages 3146–3154, 2017.
- [12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12:2825–2830, 2011.
- [13] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In Advances in Neural Information Processing Systems 30, pages 4765–4774, 2017.